

Royal Pixels Project Documentation

1. Project Overview

Royal Pixels is a premium wallpaper application built with Flutter. It features a clean architecture, Firebase integration for backend services, and a diamond-based economy system. The app provides a curated collection of high-quality 4K wallpapers with immersive parallax effects.

2. Core Layer (lib/core)

main.dart: Entry point of the app. Initializes Firebase, Riverpod, and runs the RoyalPixelsApp.

ads/ad_helper.dart: Helper for Google Mobile Ads integration.

constants/animation_constants.dart: Stores standard durations and curves for animations.

constants/app_constants.dart: General app constants (API keys, strings).

di/service_locator.dart: Sets up dependency injection using GetIt.

error/failures.dart: Defines Failure classes for error handling.

services/notification_service.dart: Manages Firebase Cloud Messaging and local notifications.

theme/app_colors.dart: Defines the app's color palette (premium dark/vibrant colors).

theme/app_theme.dart: Configures the Material theme (Dark mode).

utils/cloudinary_upload.dart: Utility for uploading images to Cloudinary.

utils/image_filter_utils.dart: Image processing utilities.

utils/royal_snack_bar.dart: Custom premium Snackbar implementation.

utils/safe_tap.dart: Debounced tap handler to prevent double clicks.

utils/screenshot_analyzer.dart: Analyzes screenshots using Google ML Kit.

widgets/login_required_sheet.dart: UI for prompting users to login.

3. Data Layer (lib/data)

datasources/auth_remote_data_source.dart: Firebase Auth interactions.

datasources/firestore_data_source.dart: Firestore CRUD operations.

models/notification_model.dart: Notification data model.

models/user_model.dart: User data model.

models/wallpaper_model.dart: Wallpaper data model.

repositories/auth_repository_impl.dart: Implementation of Auth repository.

repositories/diamond_repository_impl.dart: Implementation of Diamond (economy) repository.

repositories/notification_repository_impl.dart: Implementation of Notification repository.

repositories/payment_repository_impl.dart: Implementation of Payment repository.

repositories/wallpaper_repository_impl.dart: Implementation of Wallpaper repository.

4. Domain Layer (lib/domain)

Royal Pixels Project Documentation

entities/diamond_data.dart: Entity for diamond currency.

entities/haptic_level.dart: Enum for vibration intensity.

entities/notification_item.dart: Entity for notifications.

entities/notification_type.dart: Enum for notification categories.

entities/user_entity.dart: Core user object.

entities/wallpaper_entity.dart: Core wallpaper object.

repositories/auth_repository.dart: Auth repository interface.

repositories/diamond_repository.dart: Diamond repository interface.

repositories/notification_repository.dart: Notification repository interface.

repositories/payment_repository.dart: Payment repository interface.

repositories/wallpaper_repository.dart: Wallpaper repository interface.

usecases/add_wallpaper_usecase.dart: Use case for adding wallpapers.

usecases/buy_premium_wallpaper_usecase.dart: Handles diamond-based purchases.

usecases/delete_wallpaper_usecase.dart: Use case for deleting wallpapers.

usecases/get_wallpapers_usecase.dart: Fetches wallpaper lists.

usecases/login_usecase.dart: Handles authentication flow.

usecases/rename_category_usecase.dart: Logic for category management.

usecases/update_wallpaper_usecase.dart: Use case for modifying wallpaper details.

5. Presentation Layer (lib/presentation)

navigation/app_router.dart: GoRouter configuration and route definitions.

pages/about/about_page.dart: App info and links.

pages/auth/login_page.dart: Authentication screen.

pages/category/categories_list_page.dart: Overview of all wallpaper categories.

pages/category/category_page.dart: Grid view of wallpapers in a specific category.

pages/detail/wallpaper_detail_page.dart: Immersive detail view with parallax effects.

pages/diamond/diamond_reward_popup.dart: UI for showing earned diamonds.

pages/diamond/diamond_store_page.dart: Store for purchasing or earning diamonds.

pages/home/home_page.dart: The main landing page with featured wallpapers.

pages/home/wallpaper_search_delegate.dart: Search functionality for wallpapers.

pages/my_wallpapers/my_wallpapers_page.dart: User's uploaded or favorite wallpapers.

pages/notifications/notifications_page.dart: Notification history screen.

pages/payment/payment_bottom_sheet.dart: Interface for handling transactions.

pages/splash/splash_page.dart: Initial loading screen.

pages/subscription/subscription_page.dart: Premium membership options.

pages/upload/rename_category_page.dart: UI for editing categories.

pages/upload/upload_category_cover_page.dart: Upload UI for category thumbnails.

Royal Pixels Project Documentation

pages/upload/upload_wallpaper_page.dart: Multi-step wallpaper upload form.

providers/auth_provider.dart: Authentication state management.

providers/category_cover_provider.dart: State for category cover images.

providers/diamond_provider.dart: Manages diamond balance and rewards.

providers/download_provider.dart: Tracks download progress.

providers/favorites_provider.dart: Manages user's favorite wallpapers.

providers/guest_streak_provider.dart: Logic for daily guest rewards.

providers/haptic_provider.dart: Controls vibration feedback settings.

providers/notification_provider.dart: State for in-app notifications.

providers/parallax_provider.dart: Manages gyroscope-based parallax effect state.

providers/wallpaper_provider.dart: Core state management for wallpaper data.

widgets/diamond_counter_widget.dart: Reusable UI for showing diamond balance.

widgets/diamond_loader.dart: Premium custom loading animation.

widgets/haptic_settings_sheet.dart: UI for adjusting vibration feedback.

widgets/wallpaper_card.dart: Grid item for displaying wallpaper previews.

widgets/wallpaper_long_press_preview.dart: Interactive preview on long press.

6. Project Root & Maintenance Files

pubspec.yaml: Main configuration file for dependencies, assets, and project versioning.

analysis_options.yaml: Configures static analysis rules for code quality.

edit_logo.py: Script for processing or editing the app logo.

README.md: Project overview and setup instructions.

analysis_output.txt: Detailed log of Dart static analysis results.

analyze_machine.txt: Machine-readable analysis output for automated tools.

build_out.txt: Comprehensive log of the latest build process.

doctor.txt: Output from 'flutter doctor' verifying the development environment.

add_new_clean.txt: Log of clean architecture migration or addition of new features.

temp_git.txt: Temporary storage for git-related operations or logs.

7. Static Assets

assets/: Contains icons, logos, login backgrounds, and QR codes used in the application.